

AWS Metering Tasks: Results and Trace Analysis

Introduction	1
Results	2
Failure Mode Taxonomy	3
What the trajectories show	4
Task 1: tenant attribution	5
Task 2: entitlement overage lines	6
Task 3: usage window aggregation	7
Task 4: usage attribution chain	8
Task 5: IAM role validation	9
Conclusion	10
A Run configuration	12
B Per-trial results	12
C Run distributions	14
D Statistical detail	14
E Verified evidence	14

Introduction

This suite tests coding agents on a real AWS metering and billing codebase. Each task gives the agent a feature request in plain English, a full TypeScript repository, and a live AWS emulator it can query. The agent works alone and submits code once it’s done. A held-out verifier then runs that code against data the agent never saw and returns pass or fail. Eight independent attempts (trials) per model per task make one cell.

Task framing follows how engineers actually hand work to coding agents: a short statement of the desired behavior, not a file-by-file implementation recipe. This is the same end-to-end design principle articulated by [DeepSWE](#): the agent must locate the relevant code, resolve requirements against the repository and live environment, and decide how to implement the change. This five-task sample contains more tightly scoped tasks than the sample reported by DeepSWE: the mean prompt is 811 characters and the reference change adds 130 lines, versus 2,158 characters and 668 added lines reported by DeepSWE. All five began as real customer tickets, then received limited wording and scenario tuning for fairness and reproducibility. The graders exercise observable outputs through held-out data rather than requiring a particular helper, file layout, or implementation shape, so correct alternate solutions can pass. That combination keeps the work close to enterprise requests while testing investigation, requirement interpretation, implementation, and independent behavioral verification.

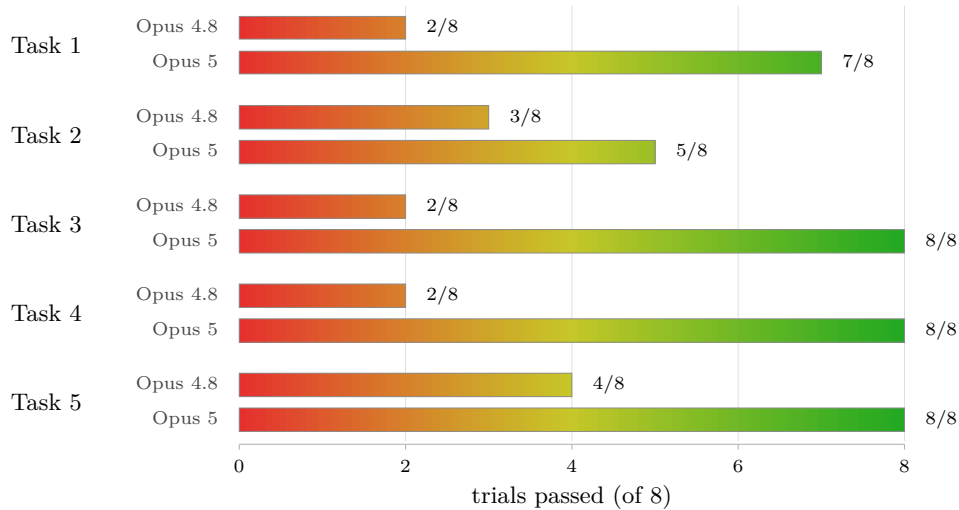
Two models ran under identical conditions, Claude Opus 4.8 and Claude Opus 5, both through Amazon Bedrock at high reasoning effort, for 80 trials in total. Two settings were revised before this run: the model response cap was raised to 32,768 tokens, and task 3’s instruction was tweaked to state its bucket geometry explicitly.

Task	What the agent must build
Task 1	split machine uptime among the right customers
Task 2	put the right overage lines on an invoice
Task 3	turn raw usage into a time-bucketed curve
Task 4	follow ownership chains across AWS services
Task 5	validate a customer’s IAM role before saving it

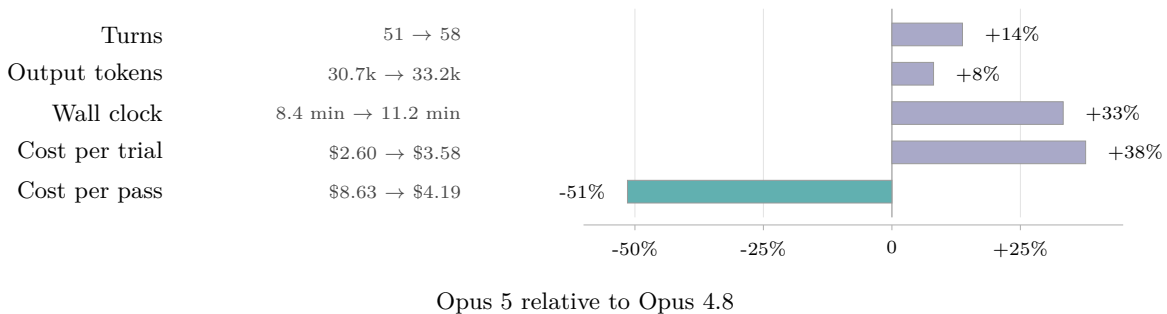
Results

Task	Opus 4.8	Opus 5	Fisher p
Task 1	2/8	7/8	0.041
Task 2	3/8	5/8	0.619
Task 3	2/8	8/8	0.007
Task 4	2/8	8/8	0.007
Task 5	4/8	8/8	0.077
Total	13/40	36/40	1.6×10^{-7}

Reading the Fisher p column. Each p is the chance of seeing a gap this large between the models if the two were in fact equally capable. Six comparisons are reported together, so the significance bar is $0.05/6 = 0.008$. The overall gap clears it by orders of magnitude, and tasks 3 and 4 clear it individually. The other tasks are not separable at eight trials per cell, which reflects the sample size rather than a finding that the models perform equally there.



Across its 40 trials Opus 5 took a median of 58 turns, 33.2k output tokens and 11.2 minutes per trial at \$3.58, against 51 turns, 30.7k tokens and 8.4 minutes at \$2.60 for Opus 4.8.



Key observations:

1. **The chart shows separation across the full suite.** Opus 5 leads on every task, by 5, 2, 6, 6, and 4 passes across tasks 1–5. The largest gaps are tasks 3 and 4 (8/8 vs. 2/8); together the five leads produce 36/40 vs. 13/40 overall ($p = 1.6 \times 10^{-7}$), rather than one outlier driving the result.
2. **Opus 5 buys its reliability with effort, and the trade pays.** It takes more turns, more tokens, and a third more wall-clock time on every attempt, costing 38% more per trial. Because that converts

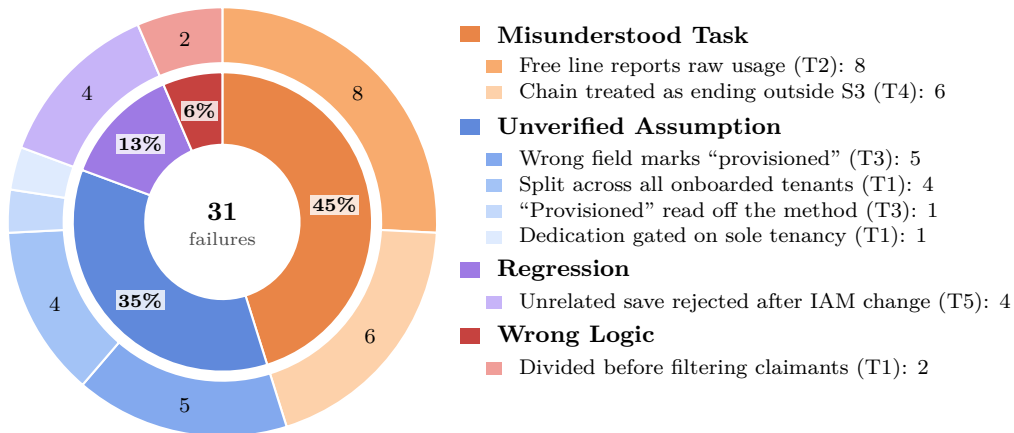
into nearly three times as many passes, a passing trial costs roughly half as much: \$4.19 against \$8.63. The traces show where the extra minutes go, into probing the emulator and checking assumptions before committing to code.

Failure Mode Taxonomy

These are the single decisive failure causes we found. The labels use [DeepSWE’s taxonomy](#) as a reference, but are adapted to the evidence available here: the instruction, repository, inspectable environment, final patch, and held-out behavior. The label describes what failed; the separate task analysis determines whether that failure is attributable to the model or to the evaluation setup. Every trial resolves to a pass or one failure case, so the counts sum to 40 per model and nothing is double-counted. Failure rows are ordered by their combined count; Pass closes the table as the denominator check. “Where” links to the task section carrying the evidence.

Label	Meaning	Opus 4.8	Opus 5	Where
Misunderstood Task	Interpreted the requested behavior incorrectly, including how rules or evidence fit together	11/40	3/40	T2, T4
Unverified Assumption	Committed to a choice without checking available evidence that could confirm or disprove it	10/40	1/40	T1, T3
Regression	Added the requested behavior but broke an existing path that should still work	4/40	0/40	T5
Wrong Logic	Used the right inputs and requirement but computed the result incorrectly	2/40	0/40	T1
Pass	Passed every held-out behavioral check	13/40	36/40	—

Each primary cause resolves further into a specific named mechanism. The inner ring below is the primary cause, and the outer ring is the mechanism behind it.

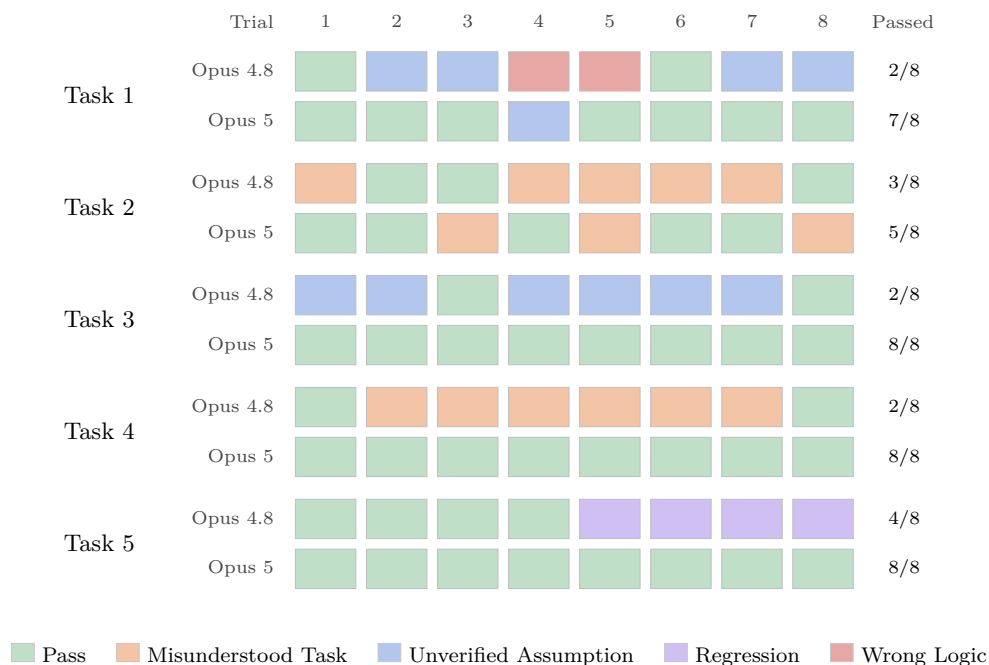


Of the 31 failures, 14 (45%) are Misunderstood Task cases: eight agents combined the allowance and zero-price rules incorrectly, while six treated unresolved ownership references as terminal instead of tracing them beyond S3. Another 11 (35%) are Unverified Assumptions, four (13%) are regressions that broke unrelated saves, and two (6%) are wrong-denominator logic errors. Opus 4.8 accounts for 27 failures, including every regression and wrong-logic case; Opus 5 has three Misunderstood Task failures and one Unverified Assumption.

Across categories, the recurring mechanism is self-confirming validation: failing trials often built tests or expected outputs from their first interpretation, allowing the same mistake to produce a green local result. Passing trials more often checked repository conventions, live emulator state, or a boundary case capable of distinguishing between implementations.

The task sections below quote multiple recorded plans, comments, executed probes, code edits, and verifier results.

The chart below shows the observed outcome of every trial. The cohort contains four distinct failure modes overall, but they cluster differently by task: Misunderstood Task accounts for all six task 4 failures, while Regression accounts for all four task 5 failures. Each square represents one rollout.



What the trajectories show

We observed the following differences between 4.8 and 5.

Opus 4.8 often submits the first plausible interpretation

On tasks 3 and 4, 12 of its 16 trials failed after the implementation stopped at an assumption the available environment could test. In task 3, six trials chose which values should carry forward without checking the offering service and seeded time series; one patch simply equated provisioned usage with `SampleType.gauge`. In task 4, six trials treated a reference missing from S3 as the end of the ownership chain; one called the remaining references “unresolvable leaves,” and none of the six queried SSM or EC2. Several then wrote tests around that first interpretation, so their local checks stayed green while the held-out verifier failed.

Opus 5 uses the environment to choose between plausible implementations

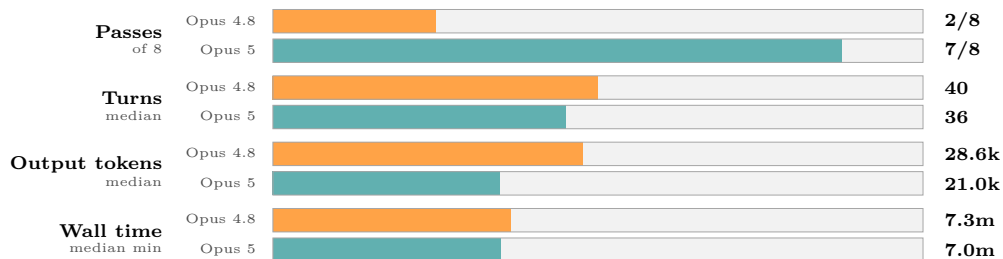
Across the same two tasks, Opus 5 passed 16 of 16 trials, compared with 4 of 16 for Opus 4.8. In task 3, passing runs checked repository conventions and the live data; trial 01 found a stored level that began before the requested window and carried it forward correctly. In task 4, all eight runs followed references across services. Trial 04 even used an emulator error listing the supported SSM calls to continue after its first API choice failed.

Opus 5 corrects risky implementations before submitting

On task 5, five Opus 5 trials first guarded IAM validation by checking whether the field name was present, then replaced that risky guard before submitting. In trial 05, the code moved from [checking whether cloudIAM was a key](#) to [checking whether it had a value](#). Another run [probed an unrelated update](#) before settling on the same rule. All eight Opus 5 trials passed task 5; four Opus 4.8 trials shipped the risky guard and rejected unrelated saves. These are visible corrections inside the recorded trajectories, not claims about hidden model cognition.

Task 1: tenant attribution

Task. Change how machine uptime is assigned to customers for billing. Assign all uptime in an account dedicated to one customer to that customer. In a shared account, use the machine’s tags together with the account’s onboarding records to find the eligible customers, divide uptime evenly among them, and leave unknown usage unbilled.



Unverified Assumption (5 failures) Four Opus 4.8 trials used the account’s entire tenant roster instead of checking the machine’s tag claims; one Opus 5 trial added a sole-tenancy condition that the available evidence did not support.

Wrong Logic (2 failures) Two Opus 4.8 trials found the tag-named customers but divided by their count before removing customers who were not onboarded, leaving the wrong denominator.

Opus 4.8 failure traces. The assumption is explicit in multiple submissions: trial 08 returns the account roster, and trial 03 divides by that roster. Two other trials reached the right candidate names but used the pre-filter count as the denominator.

```
return attribution.tenants;  
4.8 t08:1937
```

```
const share = RUNNING_TIME_INCREMENT / tenants.length;  
4.8 t03:1920
```

```
const share = RUNNING_TIME_INCREMENT / named.length;  
4.8 t04:2310
```

```
const share = RUNNING_TIME_INCREMENT / namedCustomerIds.length;  
4.8 t05:2236
```

Opus 5 comparison. Passing Opus 5 trajectories resolved the uncertainty before dividing. Trial 06 states the rule in terms of tagged customers who are genuinely onboarded; trial 05 implements the intersection directly. These are independent trajectories, not two excerpts from one run.

```
* - In a shared account the resource belongs to the tagged customers that  
O5 t06:2294
```

```
.filter((customerId) => tenancy.hasTenant(customerId));
O5 t05:2089
```

The counterexample is also visible: Opus 5 trial 04 made the report’s one Unverified Assumption for this model by adding a condition the evidence did not support.

```
* An account counts as dedicated only when exactly one customer is
O5 t04:3359
```

```
if (claims.length === 1 && tenancy[accountId].tenants.length === 1)
O5 t04:3388
```

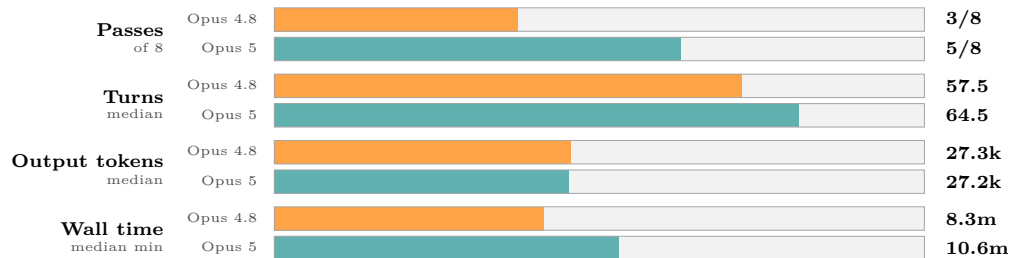
Verifier separation. The wrong roster overbilled one customer threefold; the wrong denominator underbilled the valid claimant. The cited Opus 5 trials 05 and 06 both returned [reward 1.0](#) and [reward 1.0](#).

```
monthly-uptime: cus_quarry billed 0.124999995, expected 0.041666665
verifier, 4.8 t02
```

```
monthly-uptime: cus_orbital billed 0.4027777616666666, expected 0.4444444266666666
verifier, 4.8 t04
```

Task 2: entitlement overage lines

Task. Change how customer usage becomes invoice lines. Apply each plan’s fixed usage allowance before calculating overage, charge only when the plan permits overage, and use the invoice settings to decide whether to show an invoice entry for a usage category priced at \$0, while still subtracting usage covered by the allowance.



Misunderstood Task (8 failures) Five Opus 4.8 and three Opus 5 trials applied the zero-price visibility exception to quantity, reporting raw usage instead of the allowance-adjusted amount.

Opus 4.8 failure traces. Trial 01 computes an adjusted quantity, then discards it for the free-line branch. Trial 07 makes the same choice in one ternary. These independent patches encode the same interpretation.

```
total: totalUsage,
4.8 t01:4453
```

```
total: dimensionPricedAtZero ? totalUsage : billableUsage,
4.8 t07:3825
```

Opus 5 comparison. Passing Opus 5 runs preserved the order of operations. Trial 06 states that only usage the plan permits reaches the invoice; trial 07 computes billable usage before applying the zero-price visibility rule. Both passed the held-out verifier.

```
const chargeableTotal = Offering.chargeableUsage({
O5 t06:4208
```

```
const billableUsage = Offering.billableUsageBeyondEntitlement({
O5 t07:3961
```

The stronger model was not perfect: Opus 5 trial 03 states the same wrong interpretation as the Opus 4.8 failures, which is why this task separates reliability rather than absolute capability.

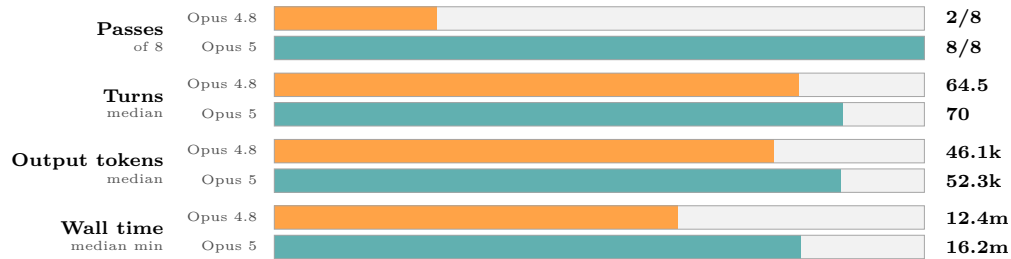
```
settings don't hide free dimensions (free lines display the total usage).
O5 t03:5392
```

Verifier separation. The held-out catalogue combines zero price with a finite allowance. Every misunderstood implementation produced the same four incorrect line quantities, while Opus 5 trials 06 and 07 returned [reward 1.0](#) and [reward 1.0](#).

```
solstice-july: cus_dunmore solstice-retention-days billed 30.0, expected 0.0
verifier, 4.8 t01
```

Task 3: usage window aggregation

Task. Change the customer portal from showing one usage total to showing how each kind of usage changes over time. Split the requested period into regular intervals anchored to standard clock boundaries, trimming the first and last intervals to the exact requested start and end times. For capacity values that remain in effect until changed, carry the last recorded value through intervals with no new reading; use zero where there was no usage, keep separately priced groups distinct, and return a value for every interval even when there are no readings.



Unverified Assumption (6 failures) All six Opus 4.8 trials got the bucket geometry right but selected a signal without checking the repository evidence: five used `gauge`, and one used the aggregation method, while the offering service and seeded series identify `SampleType.continuous`.

Opus 4.8 failure traces. Trial 01 makes the field inversion directly. Trial 07 states the same premise in prose and then uses a different unsupported heuristic. Because each run labeled its own fixtures using that premise, its new tests stayed green.

```
const provisioned = sampleType === SampleType.gauge;
4.8 t01:4653
```

```
* Currently set to gauge by default on all dimensions.
4.8 t07:2596
```

```
const provisioned = argumentAggregationMethod === aggregationMethod.last;
4.8 t07:4356
```

Opus 5 comparison. Opus 5 repeatedly grounded the choice in independent evidence. Trial 01 inspected the emulator and found a series described as a level set from before the window. Trial 08 then bound the behavior to `continuous`, the same field value used by all eight passing Opus 5 submissions.

```
dim_licences a level set a fortnight ago and moved once since
O5 t01:4924
```

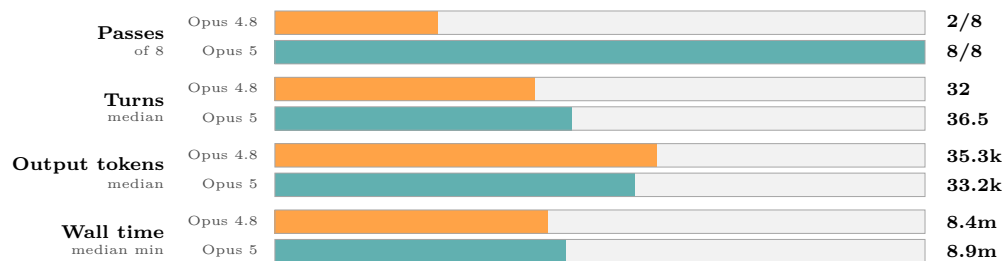
```
const provisioned = sampleType === SampleType.continuous;
O5 t08:6460
```

Verifier separation. The wrong assumption zeroed the opening bucket of a provisioned series. Opus 5 trials 01 and 08 returned [reward 1.0](#) and [reward 1.0](#); all eight Opus 5 trials passed.

```
hourly-partial-window: dim_reserved_gb bucket 0 is 0.0, expected 15.0
verifier, 4.8 t02
```

Task 4: usage attribution chain

Task. Change the spend report so every exported usage line is assigned to the owner who ultimately bears its cost, even when ownership passes through several references. Use the available code and AWS-compatible services to follow those references. Emit one row per final charge code with its quantity, money, and contributing line count; assign each amount exactly once, and collapse a repeating chain to the charge code shared by the cycle.



Misunderstood Task (6 failures) Six Opus 4.8 trials treated a reference missing from S3 as the end of the ownership chain even though the repository and live environment exposed the additional services needed to reach the final owner.

Opus 4.8 failure traces. Trial 02 turns both non-S3 reference types into an undefined result; trial 04 calls them unresolvable leaves; trial 06 explicitly assigns spend to the last intermediate owner. None of these six trials issued an SSM or EC2 API call.

```
return undefined; // alias/node dead end
4.8 t02:1515
```

```
- 'node/...' and 'alias/...' are unresolvable leaves (terminate the chain)
4.8 t04:1404
```

```
// spend rests with the last owner that carried a code.
4.8 t06:2583
```

The same runs then tested their own interpretation: trial 04's green case preserves the last resolvable code, while trial 05 declares that self-generated output to be ground truth.

```
✓ keeps the last resolvable code when the chain ends at a node
4.8 t04:2170
```



```
The full flow works and output matches the expected ground truth exactly. Spend
4.8 t05:2130
```

Opus 5 comparison. All eight Opus 5 trajectories treated unresolved references as prompts to inspect another service. Trial 04 tried the wrong SSM operation, read the emulator’s error listing the supported calls, and continued; trial 07 completed the cross-service implementation and compared its rows against an independent reconstruction.

```
ERR @aws-sdk/client-ssm InvalidAction Only GetParameter, GetParameters and
O5 t04:1165
```

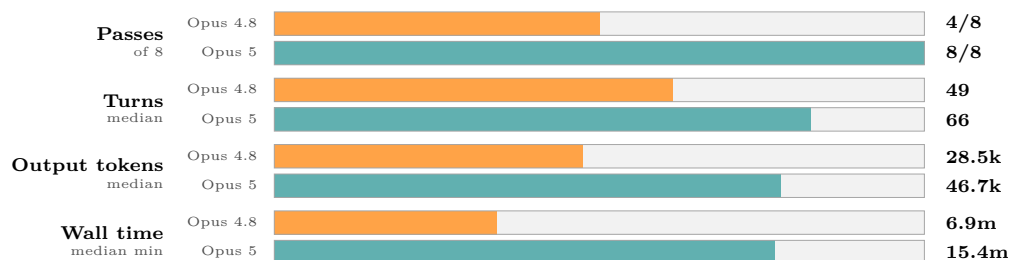
```
ROWS MATCH
O5 t07:2819
```

Verifier separation. The dead-end interpretation removed half the final charge codes and created intermediate codes with spend that should have moved onward. Opus 5 trials 04 and 07 returned [reward 1.0](#) and [reward 1.0](#); all eight Opus 5 trials passed.

```
estate: 12 of 24 charge codes absent from the report
verifier, 4.8 t04
```

Task 5: IAM role validation

Task. Change how scraper-role settings are saved so invalid AWS access is rejected before any customer settings are written. When a role is supplied, assume it with its external id and confirm that the returned credentials can read instance inventory. Reject the entire save if either check fails, treat a blank role as a disconnect that clears the external id, reject a settings block with no role, and preserve unrelated settings updates.



Regression (4 failures) All four Opus 4.8 trials added the IAM checks, but their presence guard also fired on unrelated updates and rejected requests that should still save. Because the visible unit harness and graded compile materialize optional DTO fields differently, these outcomes remain attributed to the evaluation setup.

Opus 4.8 failure traces. Trial 05 gates validation on whether the key exists; trial 06 uses `hasOwnProperty`, which is equivalent for a transformed DTO carrying an own `undefined` property. Trial 08 even reads the NodeNext configuration, but its service-level test still uses a plain object and reports a false green.

```
142 if ('cloudIAM' in updatedFiles) {
4.8 t05:2896
```

```
if (updatedFiles && Object.prototype.hasOwnProperty.call(updatedFiles,
4.8 t06:3381
```

```
"module": "NodeNext",  
4.8 t08:3745
```

```
✓ no cloudIAM in payload → unrelated update still saves  
4.8 t08:4274
```

Opus 5 comparison. Five Opus 5 trajectories initially wrote the same risky key-presence check, then changed it before submission. Trial 05 provides a direct within-run comparison: it first uses `in`, then replaces it with a value check. Trial 04 probes a transformed update and observes that an unrelated payload has no active IAM value. Trial 06 isolates the same rule in a helper that exits when the value is undefined.

```
if ('cloudIAM' in updatedFiles) {  
O5 t05:4303
```

```
if (updatedFiles?.cloudIAM !== undefined) {  
O5 t05:4357
```

```
keys [ 'logoUrl' ] has cloudIAM false  
O5 t04:4963
```

```
if (cloudIAM === undefined) {  
O5 t06:7390
```

Verifier separation. Under the graded NodeNext/ES2022 path, the Opus 4.8 guard fired on unrelated requests and returned 400 instead of saving them. The cited Opus 5 trials 04, 05, and 06 all returned [reward 1.0](#), [reward 1.0](#), and [reward 1.0](#).

```
no-cloud-block-at-all: should have been saved, answered 400  
verifier, 4.8 t05
```

Conclusion

The tasks are learnable. Every task in the suite was solved, and four of the five were solved repeatedly: Opus 5 passed 36 of 40 trials, including eight of eight on tasks 3, 4 and 5. Nothing here is a trick or a coin flip. Where a task looked intractable, the traces showed a specific reachable thing the passing trials did and the failing ones did not, whether that was intersecting two datasets before dividing, querying the service that holds the next reference, or checking which compile the graded path actually uses.

Every failure category improves with Opus 5. The gain is not concentrated in one task or one kind of mistake. It appears across the whole taxonomy:

Failure cause	Opus 4.8	Opus 5	Improvement
Misunderstood Task	11/40	3/40	+8
Unverified Assumption	10/40	1/40	+9
Regression	4/40	0/40	+4
Wrong Logic	2/40	0/40	+2

Every category improves. Opus 5 has zero regression and wrong-logic failures in this cohort, while Misunderstood Task improves by eight trials and Unverified Assumption by nine. A suite that only measured one dimension of capability would not improve this evenly.

Failures repeat within each task. Separate trials often make the same mistake and receive the same verifier

result: four wrong invoice amounts on task 2, twelve missing and fourteen extra charge codes on task 4, and one wrongly rejected save on task 5. This consistency shows that each task tests a specific behavior rather than random variation.

Where the suite should go next. The clearest limitation is coverage of model families. Both models here come from one family, so a shared habit cannot be distinguished from a general property of the task: when Opus 4.8 and Opus 5 both stop at a dead reference, we cannot yet say whether that is the task being hard or the family behaving alike. Running the same five tasks across other frontier families would separate those two explanations, and would also test whether the failure taxonomy itself transfers or whether its categories are shaped by the family we happened to measure.

A Run configuration

This section records the configuration precisely enough to reconstruct the run.

Setting	Value
Models	<code>bedrock/us.anthropic.claude-opus-4-8</code> , <code>bedrock/us.anthropic.claude-opus-5</code>
Provider	Amazon Bedrock
Reasoning effort	high, both models
Output cap	32,768 tokens on every model call
Harness	Harbor 0.18.0, mini-SWE-agent 2.4.5
Runtime	Daytona
Trials	8 per task per model, 5 tasks, 80 total
Automatic retries	disabled
Cost limit	disabled

Task identity. Every trial’s `result.json` carries its task checksum. Checksums are uniform within each task, and each matches the recorded control runs in which a reference solution scores 1.0 and a no-op agent scores 0.0. Any edit to a task moves its checksum and invalidates those controls, so both must be re-run after a change. Task 5 is stored in the repository under its original suite name, `14-iam-role-validation`, and every link in this report points at that directory.

Trial replacement. One Opus 4.8 task 2 trial timed out at the infrastructure level and was replaced by a single follow-up trial from an identical job. The replacement policy and both job specs are in the repository.

Quote verification. Every quote in this report and in the evidence appendix is machine-verified against the published files: the quote’s whitespace-normalized text must appear within three lines of its recorded location, with JSON-escape normalization inside trajectory JSON. 58 checked, none unverifiable.

The data pack. `runs.csv` gives one row per trial with reward, steps, API calls, executed tool calls, truncations, deaths, duration, output tokens, cost, and the primary and secondary labels with attribution. `manifest.json` records checksums, taxonomy definitions, a column-level data dictionary, and expected counts. `quotes.json` carries all 58 quotes with file and line. `stats.py` recomputes every number in this report from `runs.csv`, and `verify_pack.py` asserts the entire pack against the published tree in one command: every row’s reward against both recorded sources, steps and tokens and cost against the repository’s own metrics export, truncation and death flags against a fresh transcript scan, the pass table and label counts against the manifest, checksum uniformity, and all 58 quotes.

Before public release. Task 4’s build and its 16 trace directories still carry the client’s original identifiers and must be anonymized, and the separately flagged credential rotation must be completed, before this repository is published.

B Per-trial results

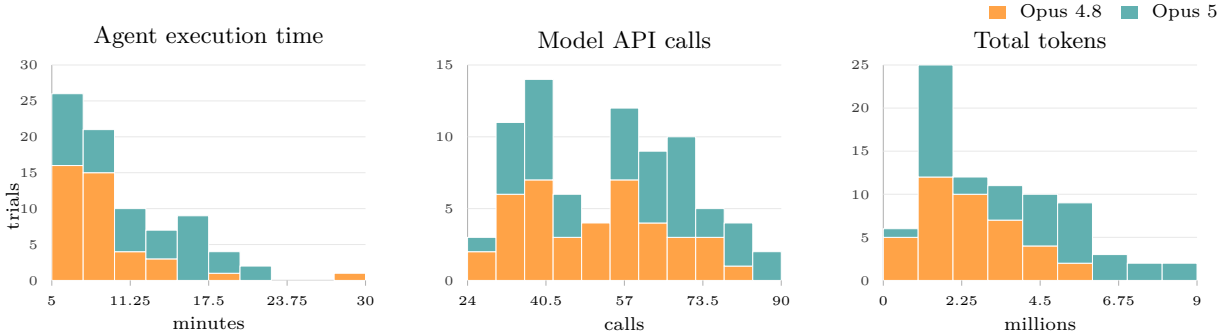
Task	Model	Trial	R	Steps	Min	Out-tok	\$	Primary label
1	Opus 4.8	01	1	36	5.7	23,952	1.51	Pass
1	Opus 4.8	02	0	32	5.9	24,697	1.57	Unverified Assumption
1	Opus 4.8	03	0	42	7.5	27,274	1.80	Unverified Assumption
1	Opus 4.8	04	0	41	7.5	30,690	1.91	Wrong Logic
1	Opus 4.8	05	0	41	9.2	38,298	2.35	Wrong Logic
1	Opus 4.8	06	1	34	7.1	29,867	1.76	Pass
1	Opus 4.8	07	0	81	29.8	36,016	3.51	Unverified Assumption
1	Opus 4.8	08	0	39	6.3	25,028	1.59	Unverified Assumption
1	Opus 5	01	1	39	9.4	29,326	2.20	Pass
1	Opus 5	02	1	35	6.3	17,141	1.36	Pass
1	Opus 5	03	1	33	6.3	20,031	1.66	Pass

Task	Model	Trial	R	Steps	Min	Out-tok	\$	Primary label
1	Opus 5	04	0	38	7.4	24,648	1.83	Unverified Assumption
1	Opus 5	05	1	37	7.2	22,490	1.67	Pass
1	Opus 5	06	1	40	7.0	20,844	1.76	Pass
1	Opus 5	07	1	27	6.2	20,660	1.31	Pass
1	Opus 5	08	1	32	7.0	21,062	1.52	Pass
2	Opus 4.8	01	0	63	9.4	30,363	3.60	Misunderstood Task
2	Opus 4.8	02	1	50	6.6	22,634	2.43	Pass
2	Opus 4.8	03	1	58	8.0	23,790	2.76	Pass
2	Opus 4.8	04	0	51	5.8	21,261	2.21	Misunderstood Task
2	Opus 4.8	05	0	68	8.8	29,796	3.47	Misunderstood Task
2	Opus 4.8	06	0	58	8.6	29,544	2.83	Misunderstood Task
2	Opus 4.8	07	0	57	9.4	35,378	3.50	Misunderstood Task
2	Opus 4.8	08	1	56	6.9	25,102	2.61	Pass
2	Opus 5	01	1	55	9.4	23,793	3.23	Pass
2	Opus 5	02	1	58	10.2	29,040	3.56	Pass
2	Opus 5	03	0	79	18.6	27,040	4.12	Misunderstood Task
2	Opus 5	04	1	66	13.9	27,746	3.61	Pass
2	Opus 5	05	0	63	10.1	25,100	3.24	Misunderstood Task
2	Opus 5	06	1	81	22.3	30,782	4.61	Pass
2	Opus 5	07	1	57	9.8	25,392	3.13	Pass
2	Opus 5	08	0	66	11.0	27,340	3.68	Misunderstood Task
3	Opus 4.8	01	0	67	10.7	43,010	4.06	Unverified Assumption
3	Opus 4.8	02	0	58	11.6	42,367	3.20	Unverified Assumption
3	Opus 4.8	03	1	77	14.7	55,365	5.01	Pass
3	Opus 4.8	04	0	67	12.0	49,003	4.34	Unverified Assumption
3	Opus 4.8	05	0	62	9.8	38,481	3.69	Unverified Assumption
3	Opus 4.8	06	0	74	18.6	59,821	5.71	Unverified Assumption
3	Opus 4.8	07	0	60	12.8	43,134	4.11	Unverified Assumption
3	Opus 4.8	08	1	59	14.0	50,887	4.42	Pass
3	Opus 5	01	1	80	16.4	54,129	6.37	Pass
3	Opus 5	02	1	88	17.6	56,813	7.25	Pass
3	Opus 5	03	1	64	13.3	44,847	4.92	Pass
3	Opus 5	04	1	67	15.2	50,215	5.55	Pass
3	Opus 5	05	1	70	16.1	54,917	6.73	Pass
3	Opus 5	06	1	61	13.2	44,694	4.47	Pass
3	Opus 5	07	1	84	19.2	64,469	7.77	Pass
3	Opus 5	08	1	70	16.3	50,459	6.12	Pass
4	Opus 4.8	01	1	41	9.9	42,277	2.59	Pass
4	Opus 4.8	02	0	29	8.4	34,023	1.90	Misunderstood Task
4	Opus 4.8	03	0	27	7.2	29,276	1.71	Misunderstood Task
4	Opus 4.8	04	0	32	6.3	27,717	1.62	Misunderstood Task
4	Opus 4.8	05	0	32	9.1	35,843	1.98	Misunderstood Task
4	Opus 4.8	06	0	32	8.3	36,037	2.14	Misunderstood Task
4	Opus 4.8	07	0	33	8.4	34,895	1.82	Misunderstood Task
4	Opus 4.8	08	1	51	8.8	35,620	2.76	Pass
4	Opus 5	01	1	35	7.2	28,956	2.21	Pass
4	Opus 5	02	1	36	9.1	33,564	2.46	Pass
4	Opus 5	03	1	44	10.3	38,151	2.59	Pass
4	Opus 5	04	1	43	11.4	41,706	3.01	Pass
4	Opus 5	05	1	36	8.8	33,553	2.21	Pass
4	Opus 5	06	1	42	9.9	32,775	2.53	Pass
4	Opus 5	07	1	34	6.9	29,120	2.01	Pass
4	Opus 5	08	1	37	7.1	29,023	2.11	Pass
5	Opus 4.8	01	1	58	8.4	33,841	2.88	Pass
5	Opus 4.8	02	1	52	6.9	30,101	2.58	Pass
5	Opus 4.8	03	1	73	12.5	51,828	4.87	Pass
5	Opus 4.8	04	1	61	9.5	30,669	3.05	Pass
5	Opus 4.8	05	0	40	6.0	24,004	1.89	Regression
5	Opus 4.8	06	0	37	6.1	21,241	1.92	Regression
5	Opus 4.8	07	0	43	6.1	24,291	2.12	Regression
5	Opus 4.8	08	0	46	6.8	26,887	2.64	Regression
5	Opus 5	01	1	69	15.1	46,561	5.31	Pass
5	Opus 5	02	1	64	15.5	46,866	4.96	Pass
5	Opus 5	03	1	58	16.1	51,455	4.67	Pass
5	Opus 5	04	1	73	21.1	53,990	5.78	Pass
5	Opus 5	05	1	62	14.9	42,473	4.45	Pass
5	Opus 5	06	1	75	16.5	55,766	6.15	Pass
5	Opus 5	07	1	54	12.5	45,354	4.23	Pass
5	Opus 5	08	1	68	15.3	37,951	4.90	Pass

All 80 trials; the trial number links to its trajectory directory. R is the binary verifier reward, Steps counts assistant messages, Min is agent wall-clock, Out-tok is completion tokens, and \$ is model spend. Truncation and death columns are omitted because every value is zero across all 80 trials; they remain in `runs.csv` along with API-call and tool-call counts.

C Run distributions

Every trial finished with a submission. The charts below show how much work the tasks required; no trial was cut off by a limit. Bars are stacked by model.



Most trials finish in under 15 minutes using fewer than 70 model calls and under 4 million total tokens, where the total counts every prompt token the agent re-sent as its context grew, not just what it generated. The tails are modest and have ordinary explanations: the longest run, just under 30 minutes, is an Opus 4.8 task 1 trial, and the heaviest token counts belong to task 3, the task that requires the most reading before any code can be written. Nothing in these distributions is truncated, and no trial approached the 32,768-token response cap on any single call.

D Statistical detail

The metric is 8 independent single rollouts per cell, with a binary reward from the held-out verifier. Counts phrased “of 8” and “of 40” include every rollout; no trial died or submitted empty, so no attempted-versus-completed distinction applies.

Wilson 95% intervals: Opus 4.8 total 13/40 (20–48%), Opus 5 total 36/40 (77–96%). Per cell at $n=8$: 2/8 spans 7–59%, 4/8 spans 22–78%, 8/8 spans 68–100%. These are wide by construction, so single-cell differences of one or two trials should not be read as meaningful on their own.

Two-sided Fisher exact tests comparing the models, a stated family of six: totals $p = 1.6 \times 10^{-7}$; task 3 $p = 0.0070$; task 4 $p = 0.0070$; task 1 $p = 0.0406$; task 5 $p = 0.0769$; task 2 $p = 0.6193$. Under Bonferroni ($\alpha/6 = 0.0083$) the totals and tasks 3 and 4 remain significant, task 1 is suggestive, and tasks 5 and 2 are not individually significant at $n=8$. A continuity-corrected Cochran-Mantel-Haenszel test stratified by task confirms the overall gap is not an artifact of pooling ($p = 7.0 \times 10^{-7}$).

The per-mode claims in the task sections are censuses rather than samples. Statements such as “0 of 6 failing trials issued an SSM call” or “16 of 16 trials implemented clamped buckets” are directly observed properties of every relevant trial and carry no sampling error, so the interval widths above do not apply to them. `stats.py` in the data pack recomputes everything in this appendix from `runs.csv`.

E Verified evidence

Task	Location	Quote
1	4.8 t03:1920	const share = RUNNING_TIME_INCREMENT / tenants.length;
1	4.8 t08:1937	return attribution.tenants;
1	verifier, 4.8 t02	monthly-uptime: cus_quarry billed 0.124999995, expected 0.041666665
1	4.8 t04:2310	const share = RUNNING_TIME_INCREMENT / named.length;
1	4.8 t05:2236	const share = RUNNING_TIME_INCREMENT / namedCustomerIds.length;
1	verifier, 4.8 t04	monthly-uptime: cus_orbital billed 0.4027777616666666, expected 0.4444444266666666
1	O5 t04:3388	if (claims.length === 1 && tenancy[accountId].tenants.length === 1)
1	O5 t04:3359	* An account counts as dedicated only when exactly one customer is
1	verifier, O5 t04	monthly-uptime: cus_orbital billed 0.3611110966666667, expected 0.4444444266666666
1	O5 t06:2294	* - In a shared account the resource belongs to the tagged customers that
1	O5 t05:2089	.filter((customerId) => tenancy.hasTenant(customerId));
1	4.8 t01:2200	TypeError: registry.loadCustomers is not a function
2	4.8 t07:3825	total: dimensionPricedAtZero ? totalUsage : billableUsage,
2	O5 t08:4685	const billableUsage = isFreeDimension ? usageTotal :
2	O5 t03:5392	settings don't hide free dimensions (free lines display the total usage).
2	O5 t06:4208	const chargeableTotal = Offering.chargeableUsage({
2	4.8 t08:3719	const owesSomething = billableUsage > 0 && unitCost > 0;
2	O5 t07:3961	const billableUsage = Offering.billableUsageBeyondEntitlement({
2	O5 t08:5240	test('a dimension priced at zero is shown with its usage when free
2	4.8 t07:4888	The output exactly matches the requirements. Let me remove the repro spec and
2	O5 t05:4887	quantity: 10, unitCost: 0 }]);
2	4.8 t08:3706	} else if (dimension.overageAllowed !== overageAllowedEnum.true)
2	O5 t05:5229	if (!overageAllowed) {
2	O5 t02:5051	✓ overage unspecified charges nothing
2	4.8 t01:4453	total: totalUsage,
2	verifier, 4.8 t01	solstice-july: cus_dunmore solstice-retention-days billed 30.0, expected 0.0
3	4.8 t01:4633	const provisioned = sampleType === SampleType.gauge;
3	reference solution	+ const level = sampleType === SampleType.continuous;
3	verifier, 4.8 t02	hourly-partial-window: dim_reserved_gb bucket 0 is 0.0, expected 15.0
3	4.8 t07:4356	const provisioned = argumentAggregationMethod === aggregationMethod.last;
3	4.8 t07:2596	* Currently set to gauge by default on all dimensions.
3	verifier, 4.8 t07	hourly-partial-window: dim_reserved_gb bucket 0 is 0.0, expected 15.0
3	4.8 t08:4175	const start = edge.getTime() < startTime.getTime() ? startTime : edge;
3	4.8 t08:4314	await readUsageSamples({ series, startTime: new Date(0),
3	O5 t01:4924	dim_licences a level set a fortnight ago and moved once since
3	O5 t08:6460	const provisioned = sampleType === SampleType.continuous;
4	4.8 t04:1404	- 'node/...' and 'alias/...' are unresolvable leaves (terminate the chain)
4	4.8 t02:1515	return undefined; // alias/node dead end
4	4.8 t06:2583	// spend rests with the last owner that carried a code.
4	4.8 t04:2170	✓ keeps the last resolvable code when the chain ends at a node
4	4.8 t05:2130	The full flow works and output matches the expected ground truth exactly.
4	4.8 t07:1592	Spend
4	4.8 t08:1592	await put('nodes/acct/us-east-1/i-1.json', { chargeCode:
4		'alias/*' → SSM param, 'node/*' → EC2 instance tags. Let me look at the
4		rest and
4	O5 t04:1165	ERR @aws-sdk/client-ssm InvalidAction Only GetParameter, GetParameters and
4	O5 t07:2819	ROWS MATCH
4	verifier, 4.8 t04	estate: 12 of 24 charge codes absent from the report
5	4.8 t05:2896	142 if ('cloudIAM' in updatedFiles) {
5	4.8 t06:3381	if (updatedFiles && Object.prototype.hasOwnProperty.call(updatedFiles,
5	verifier, 4.8 t05	no-cloud-block-at-all: should have been saved, answered 400
5	4.8 t06:3684	✓ does not validate when cloudIAM is absent (2 ms)
5	4.8 t08:4274	✓ no cloudIAM in payload → unrelated update still saves
5	4.8 t08:3745	"module": "NodeNext",
5	4.8 t04:4637	if (updatedFiles.cloudIAM !== undefined) {
5	O5 t06:7390	if (cloudIAM === undefined) {
5	O5 t06:3554	meteringco-reports-reader ext=nw-a01b: ASSUME_OK DESCRIBE_ERR
5	O5 t05:4303	if ('cloudIAM' in updatedFiles) {
5	O5 t05:4357	if (updatedFiles?.cloudIAM !== undefined) {
5	O5 t04:4963	keys ['logoUrl'] has cloudIAM false